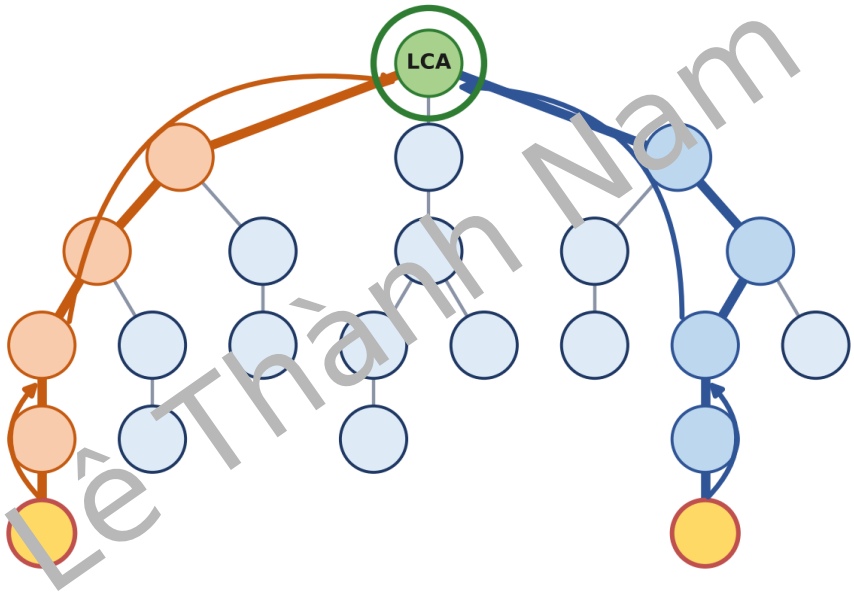
Tổng hợp và biên soạn: Lê Thành Nam

TÀI LIỆU BỒI DƯỠNG HỌC SINH GIỎI TIN HỌC

THUẬT TOÁN TRÊN CÂY

Dễ hiểu cho người bắt đầu từ con số 0

TẬP 2 · TỔ TIÊN CHUNG GẦN NHẤT (LCA)



Lê Thành Nam

Bảng nhảy · Euler tour và bảng thưa · Tarjan · Chia nặng nhẹ · Farach-Colton – Bender · Thang

Chương 1. BẢNG NHẢY NHỊ PHÂN
Chương 2. EULER TOUR VÀ BẢNG THƯA
Chương 3. LCA NGOẠI TUYẾN
Chương 4. CHIA NẶNG NHẸ
Chương 5. ỨNG DỤNG CỦA LCA
Chương 6. NÂNG CAO: O(1) THẬT SỰ

LỜI NÓI ĐẦU

Trong các bài toán trên cây, câu hỏi "hai đỉnh này gặp nhau ở đâu khi cùng đi về gốc?" xuất hiện nhiều đến mức nó có một cái tên riêng: **tổ tiên chung gần nhất**, viết tắt **LCA** (Lowest Common Ancestor). Khoảng cách giữa hai đỉnh, giá trị lớn nhất trên đường đi, số lượt đi qua mỗi đỉnh, ... hầu như mọi câu hỏi về **đường đi** trên cây đều bắt đầu bằng việc tìm LCA.

Cuốn sách này là **Tập 2: Tổ tiên chung gần nhất** của bộ sách về thuật toán trên cây, tiếp nối Tập 1 (Nền tảng). Em cần đã quen với các kiến thức của Tập 1: danh sách kề, mảng cha, BFS, DFS không đệ quy, kích thước cây con, tin/tout. Những kiến thức ấy được nhắc lại ngắn gọn ở Phần mở đầu.

Giống Tập 1, mỗi bài được chia làm hai phần:

- **Phần A – Lí thuyết:** trình bày thẳng vào kiến thức, **chi tiết hơn Tập 1**: định nghĩa, tính chất, định lí kèm chứng minh từng ý (mỗi ý một hình), thuật toán, chạy tay từng bước nhỏ trên cây mẫu T14, code mẫu, so sánh với các phương pháp khác, độ phức tạp thời gian và bộ nhớ, bẫy thường gặp.
- **Phần B – Bài vận dụng:** một bài toán theo khuôn mẫu thi giải đầy đủ: ý tưởng ngây thơ với sơ đồ khối và chương trình vét cạn, chỉ ra bế tắc, áp dụng lí thuyết để phá bế tắc, chạy tay, chương trình hoàn chỉnh, mở rộng và luyện tập.

Sách gồm **Phần mở đầu** (định nghĩa và tính chất của LCA, bản đồ tập sách, cách chọn phương pháp) và **13 bài học chia thành 6 chương**:

- **Chương 1. Bảng nhảy nhị phân:** tổ tiên thứ k, LCA, max/min trên đường đi.
- **Chương 2. Euler tour và bảng thưa:** RMQ bằng bảng thưa, LCA trong O(1).
- **Chương 3. LCA ngoại tuyến:** cấu trúc các tập rời nhau (DSU), thuật toán Tarjan.
- **Chương 4. Chia nặng nhẹ:** chuỗi nặng, định lí log n, LCA với bộ nhớ O(n).
- **Chương 5. Ứng dụng của LCA:** khoảng cách và đường đi, hiệu tiền tố trên cây.
- **Chương 6. Nâng cao:** Farach-Colton – Bender (O(n) chuẩn bị, O(1) mỗi truy vấn), tổ tiên thứ k trong O(1) bằng thang.

Các bài vận dụng gồm những bài kinh điển: **CSES Company Queries I, II** (Bài 1, 2), **CSES Distance Queries** (Bài 10), **CSES Counting Paths** (Bài 11), được đặt lại trong bối cảnh nhà trường. Mọi chương trình đều đã được đối chiếu với chương trình vét cạn trên hàng trăm cây ngẫu nhiên, và chạy thử với dữ liệu lớn nhất (kể cả cây đường thẳng); thời gian đo được ghi ngay trong sách để em so sánh các phương pháp bằng con số thật.

Chương trình viết bằng C++17, đọc và ghi tệp theo kiểu "có tệp thì đọc tệp", dùng được với Themis lẫn các trang chấm trực tuyến. Mọi chương trình đều không có chú thích bên trong; các dòng quan trọng được giải thích ngay bên dưới.

Chúc các em và các thầy cô học tập say mê, dạy tốt và đạt kết quả cao!

Lê Thành Nam

Lê Thành Nam

CÁCH HỌC VỚI TÀI LIỆU NÀY

Phần	Nội dung
Tóm tắt bài học	Khung tóm tắt ở đầu bài: kiến thức chính, khuôn cài đặt, độ phức tạp, khi nào dùng, bài vận dụng.
Phần A – Lí thuyết	Định nghĩa, tính chất, định lí kèm chứng minh từng ý; thuật toán; chạy tay từng bước nhỏ trên hình; code mẫu; độ phức tạp; bẫy; ghi nhớ.
Phần B – Bài vận dụng	Đề bài như đề thi thật: dữ liệu vào, kết quả, giới hạn, ví dụ.
B1. Ý tưởng ngây thơ	Cách làm ai cũng nghĩ ra: sơ đồ khối, chạy tay, chương trình vét cạn, đếm số phép tính.
Bế tắc	Chỉ ra chính xác vì sao cách ngây thơ quá chậm hoặc quá tốn bộ nhớ.
B2. Phá bế tắc	Áp dụng lí thuyết Phần A, từng bước, chạy tay trên hình.
B3. Chương trình hoàn chỉnh	Chương trình C++ đã kiểm chứng, giải thích các dòng quan trọng.
B4. Mở rộng, Luyện tập	Các biến thể thường gặp; bài luyện thêm và câu tự kiểm tra có đáp số.

Các khung màu trong sách có ý nghĩa cố định:

Khung	Ý nghĩa
TÓM TẮT BÀI HỌC	Tóm tắt toàn bộ bài ở ngay đầu bài.
ĐỊNH NGHĨA	Định nghĩa chính xác một khái niệm.
TÍNH CHẤT / ĐỊNH LÍ	Một khẳng định quan trọng, luôn có chứng minh đi kèm.
THUẬT TOÁN	Các bước của một thuật toán.
HỘP CÔNG CỤ C++	Giới thiệu một công cụ C++ khi nó được dùng lần đầu.
ĐẾM SỐ PHÉP TÍNH	Phân tích độ phức tạp: vì sao chậm, vì sao nhanh.
BẾ TẮC	Chỗ cách làm ngây thơ không vượt qua được.
BẪY THƯỜNG GẶP	Các lỗi hay gặp.
GHI NHỚ	Tóm tắt cuối phần lí thuyết.

Quy ước màu trong các hình được giữ nguyên trong cả cuốn sách: **đỉnh vàng viền đỏ** là đỉnh đang xét; **đỉnh xanh lá** là đỉnh đã xử lí xong (hoặc là lá, tùy hình); **đỉnh xanh dương nhạt** là đỉnh bình thường; **đỉnh viền nét đứt** là đỉnh chưa được thăm; **đỉnh tím** là đỉnh dùng trong chứng minh; **gốc** tô xanh đậm; **mũi tên cong** là một bước nhảy (nhãn trên mũi tên là số bước hoặc tên bảng). Số nhỏ màu đỏ cạnh mỗi đỉnh là giá trị đang tính.

Lời khuyên khi học: Hãy đọc với giấy và bút bên cạnh. Gặp một hình chạy tay, hãy che phần lời giải lại, tự làm bước đó rồi mới đọc. Gặp một định lí, hãy tự vẽ một cái cây khác và kiểm tra định lí trên cây của mình. Chỉ khi tự viết lại được chương trình và tự giải thích được vì sao nó đúng mà không nhìn sách, em mới thật sự nắm được bài.

Lê Thành Nam

MỤC LỤC

LỜI NÓI ĐẦU **2**
CÁCH HỌC VỚI TÀI LIỆU NÀY **3**
PHẦN MỞ ĐẦU. TỔ TIÊN CHUNG GẦN NHẤT VÀ BẢN ĐỒ TẬP 2 ..... **6**
CHƯƠNG 1. BẢNG NHẢY NHỊ PHÂN **15**
Bài 1. Tổ tiên thứ k và bảng nhảy nhị phân 16
Bài 2. LCA bằng bảng nhảy nhị phân 38
Bài 3. Thông tin trên đường đi bằng bảng nhảy 55
CHƯƠNG 2. EULER TOUR VÀ BẢNG THƯA **73**
Bài 4. Bảng thưa (Sparse Table) 74
Bài 5. LCA bằng Euler tour và bảng thưa 89
CHƯƠNG 3. LCA NGOẠI TUYẾN **105**
Bài 6. Cấu trúc các tập rời nhau (DSU) 106
Bài 7. Thuật toán Tarjan ngoại tuyến 121
CHƯƠNG 4. CHIA NẶNG NHẸ **134**
Bài 8. Chia nặng nhẹ (Heavy-Light Decomposition) 135
Bài 9. LCA bằng chia nặng nhẹ 148
CHƯƠNG 5. ỨNG DỤNG CỦA LCA **160**
Bài 10. Khoảng cách và đường đi trên cây 161
Bài 11. Hiệu tiền tố trên cây 174
CHƯƠNG 6. NÂNG CAO: O(1) THẬT SỰ **187**
Bài 12. Thuật toán Farach-Colton – Bender 188
Bài 13. Tổ tiên thứ k trong O(1): phân rã thang 201
TỔNG KẾT: CHỌN CÔNG CỤ LCA CHO TỪNG ĐỀ BÀI **215**

Lê Thành Nam

PHẦN MỞ ĐẦU

TỔ TIÊN CHUNG GẦN NHẤT VÀ BẢN ĐỒ TẬP 2

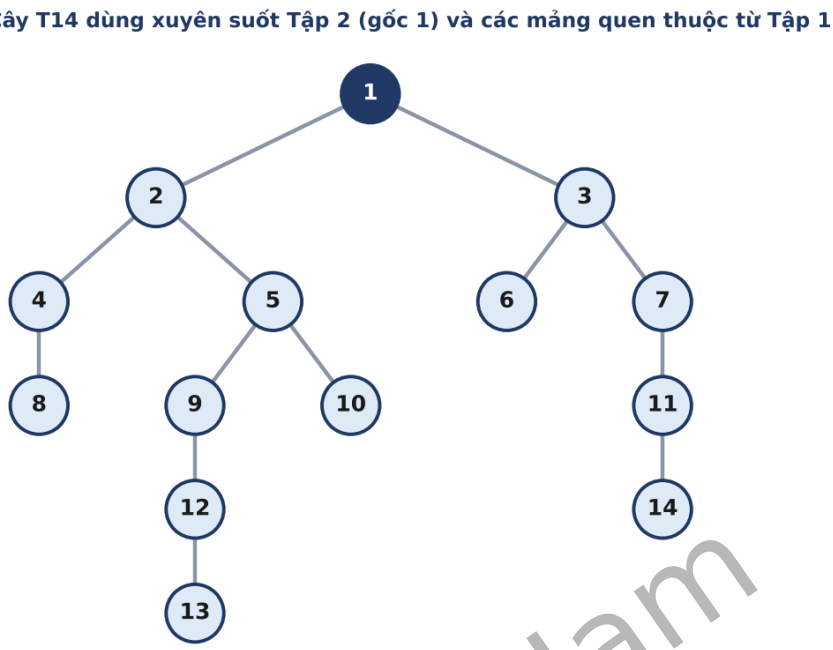
Phần này nhắc lại những gì cần từ Tập 1, định nghĩa chính xác tổ tiên chung gần nhất (LCA), chứng minh các tính chất sẽ dùng nhiều lần, trình bày hai cách ngây thơ, và vẽ bản đồ sáu chương phía sau cùng cách chọn phương pháp phù hợp với từng đề bài.

1. Nhắc lại từ Tập 1

Trong cả tập sách, cây luôn được **treo ở gốc 1**. Ta dùng lại các kí hiệu và các cách duyệt của Tập 1:

Kí hiệu	Ý nghĩa	Cách tính (Tập 1)
par[v]	cha của v; par[1] = 0	đọc trực tiếp, hoặc BFS từ gốc nếu cho dạng cạnh (Bài 3)
dep[v]	độ sâu: số cạnh từ gốc tới v	BFS, dep[con] = dep[cha] + 1 (Bài 2, 3)
ord	thứ tự BFS của u, u đứng trước con	BFS (Bài 3)
sz[v]	số đỉnh trong cây con của v	duyệt ord ngược (Bài 4)
tin[v], tout[v]	thời điểm vào; số lớn nhất trong cây con	DFS không đệ quy (Bài 5, 6)
u tổ tiên của v	tin[u] ≤ tin[v] ≤ tout[u]	O(1) (Bài 6)

Lê Thanh Nam



v	1	2	3	4	5	6	7	8	9	10	11	12	13	14
par	0	1	1	2	2	3	3	4	5	5	7	9	12	11
dep	0	1	1	2	2	2	3	3	3	3	4	5	4	
tin	0	1	9			10	11	3	5	8	12	6	7	13
tout	13	8	13	3	8	10	13	3	7	8	13	7	7	13

Lê Thành Nam

Hình 0.1. *Cây mẫu T14 và các mảng của nó*

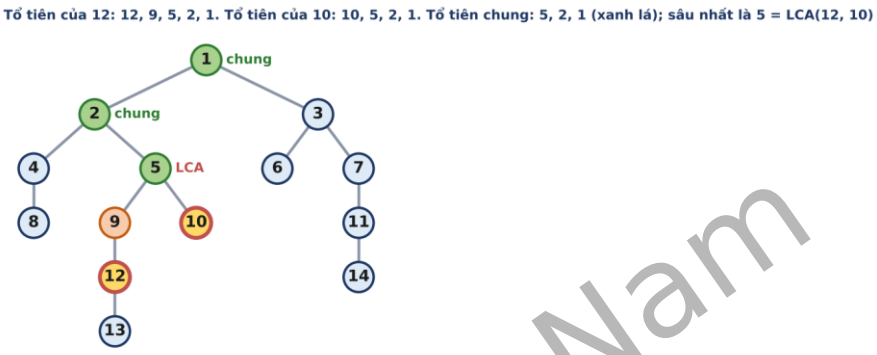
Cây T14 có 14 đỉnh, sâu nhất là 5 (đỉnh 13). Nó đủ nhỏ để chạy tay, đủ sâu để thấy rõ cái lợi của việc "nhảy xa". Hầu hết các hình trong sách dùng cây này, nên em hãy chép nó ra giấy và giữ bên cạnh khi đọc.

> **VÌ SAO KHÔNG DÙNG ĐỆ QUY?**
>
> Mọi chương trình trong sách duyệt cây **không đệ quy** (BFS, hoặc DFS bằng ngăn xếp tự quản lí), vì cây đường thẳng $2 \cdot 10^5 – 10^6$ đỉnh làm đệ quy tràn ngăn xếp trên nhiều máy chấm (Tập 1, Bài 5).

2. Tổ tiên chung gần nhất

ĐỊNH NGHĨA

- Quy ước: mỗi đỉnh là **tổ tiên của chính nó**.
- w là **tổ tiên chung** của u và v nếu w là tổ tiên của u và là tổ tiên của v.
- **LCA(u, v)** (Lowest Common Ancestor, tổ tiên chung gần nhất) là tổ tiên chung có **độ sâu lớn nhất**.



Hình 0.2. *Tổ tiên chung và tổ tiên chung gần nhất*

Các tính chất cơ bản

T1. TỒN TẠI VÀ DUY NHẤT

Gốc là tổ tiên chung của mọi cặp, nên tổ tiên chung luôn tồn tại. Các tổ tiên của u là các đỉnh trên đường u → gốc, **mỗi độ sâu đúng một đỉnh**; vì vậy tổ tiên chung có độ sâu lớn nhất là duy nhất.

T2. CÁC TỔ TIÊN CHUNG LÀ L VÀ TỔ TIÊN CỦA L

Với L = LCA(u, v): w là tổ tiên chung của u, v **khi và chỉ khi** w là tổ tiên của L. Chứng minh: hai đường u → gốc và v → gốc gặp nhau lần đầu ở L; từ L trở lên chúng trùng nhau (cùng là đường L → gốc), dưới L chúng tách rời.

T3. TỔ TIÊN

LCA(u, v) = u khi và chỉ khi u là tổ tiên của v. Đặc biệt LCA(u, u) = u.

T4. ĐƯỜNG ĐI

Đường đi duy nhất giữa u và v là: đi từ u **lên** L, rồi từ L **xuống** v. Do đó d(u, v) = dep[u] + dep[v] − 2·dep[L] (chứng minh chi tiết ở Bài 10).

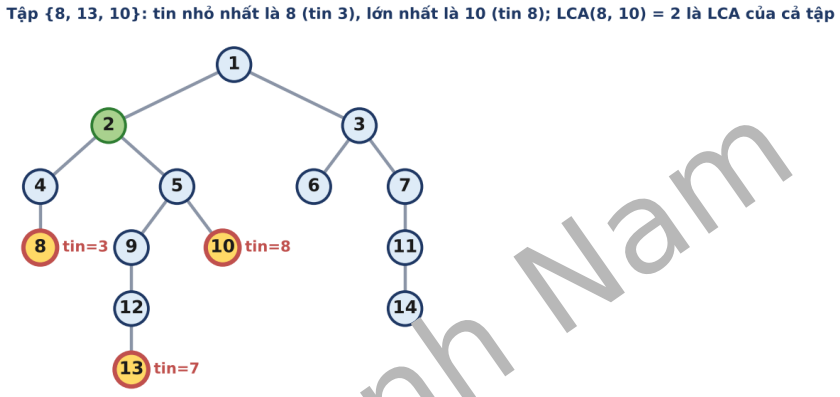
Lê Thành Nam

T5. GIAO HOÁN VÀ KẾT HỢP

LCA(u, v) = LCA(v, u) và **LCA(a, LCA(b, c)) = LCA(LCA(a, b), c)**: cả hai vế đều là tổ tiên chung sâu nhất của ba đỉnh a, b, c (theo T2, tổ tiên chung của ba đỉnh chính là tổ tiên chung của một đỉnh với LCA của hai đỉnh kia). Vì vậy LCA của cả một tập đỉnh được định nghĩa rõ ràng.

T6. LCA CỦA MỘT TẬP

LCA của một tập đỉnh bằng **LCA của hai đỉnh có tin nhỏ nhất và tin lớn nhất** trong tập.

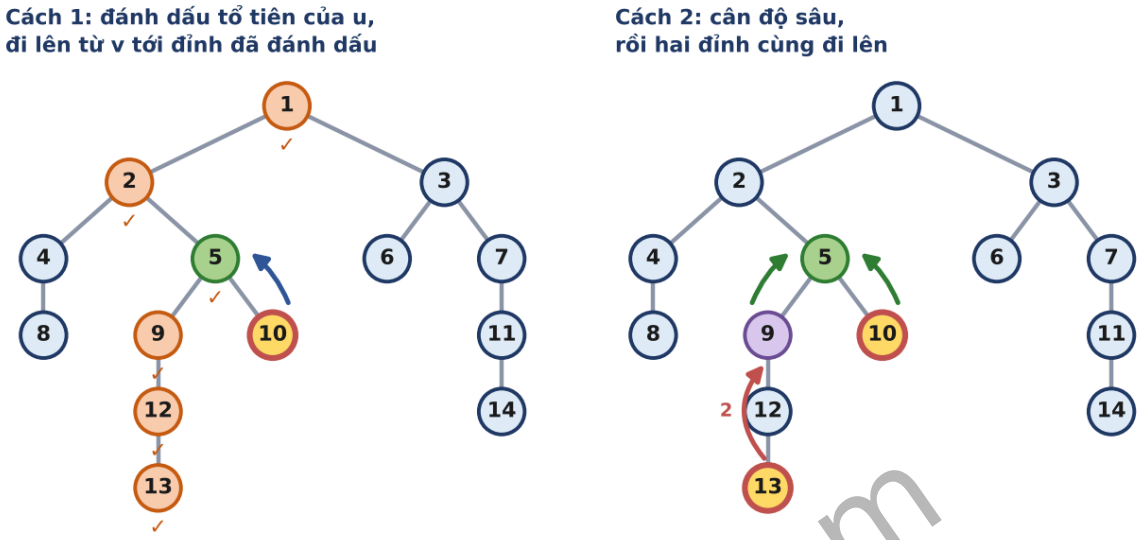


***Hình 0.3** – LCA của tập {8, 13, 10}*

Chứng minh T6: gọi a, b là hai đỉnh có tin nhỏ nhất và lớn nhất, L = LCA(a, b). Mọi đỉnh x của tập có tin[a] ≤ tin[x] ≤ tin[b]. Vì a và b cùng thuộc cây con của L, các số tin từ tin[a] tới tin[b] đều nằm trong đoạn [tin[L], tout[L]] (Tập 1, Bài 6), nên x cũng thuộc cây con của L: L là tổ tiên chung của cả tập. Một tổ tiên chung sâu hơn của cả tập sẽ là tổ tiên chung của a và b sâu hơn L, mâu thuẫn.

Lê Thành Nam

3. Hai cách ngây thơ



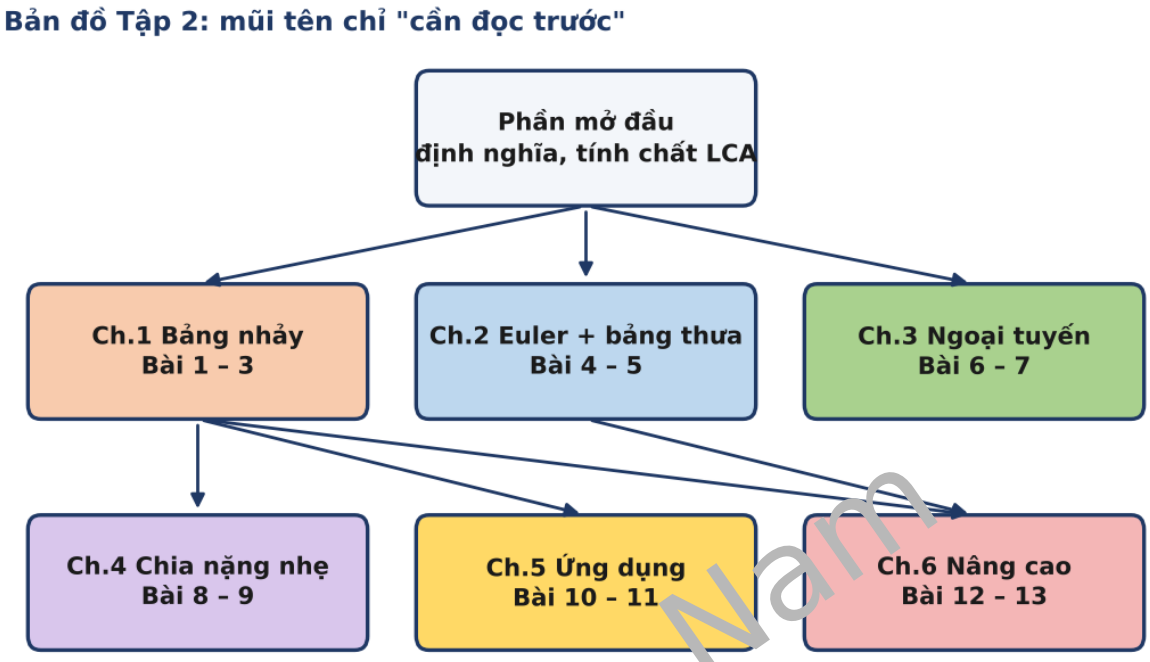
Hình 0.4. *Hai cách tìm LCA không cần chuẩn bị*

- **Cách 1 – đánh dấu:** đánh dấu u và mọi tổ tiên của u, đi lên từ v tới đỉnh đầu tiên đã được đánh dấu. Đúng theo T2.
- **Cách 2 – cân độ sâu:** đưa đỉnh sâu hơn lên cho bằng độ sâu đỉnh kia, rồi cho hai đỉnh cùng đi lên từng bước tới khi trùng nhau. Đúng theo bổ đề đơn điệu ở Bài 2.

Cả hai cách tốn **O(dep[u] + dep[v])** mỗi câu hỏi, tới O(n) với cây đường thẳng. Với q câu hỏi là O(n·q): $4 \cdot 10^{10}$ bước khi n = q = $2 \cdot 10^5$. Toàn bộ Tập 2 là câu chuyện làm cho mỗi câu hỏi nhanh hơn, bằng nhiều con đường khác nhau.

Lê Thành Nam

4. Bản đồ Tập 2



Hình 0.5. *Sáu chương và thứ tự đọc*

Bài	Kiến thức	Độ phức tạp	Bài vận dụng
1	**Tổ tiên thứ k và bảng nhảy nhị phân**	Chuẩn bị O(n log n), mỗi truy vấn O(log n), bộ nhớ O(n log n)	Cấp trên thứ k (CSES Company Queries I) [CAPTRENK]
2	**LCA bằng bảng nhảy nhị phân**	O(n log n) chuẩn bị, O(log n) mỗi truy vấn	Sếp chung (CSES Company Queries II) [SEPCHUNG]
3	**Thông tin trên đường đi bằng bảng nhảy**	O(n log n) chuẩn bị, O(log n) mỗi truy vấn	Đoạn đường dốc nhất [DOCNHAT]
4	**Bảng thưa (Sparse Table)**	O(n log n) chuẩn bị, O(1) mỗi câu hỏi	Chênh lệch nhiệt độ [NHIETDO]
5	**LCA bằng Euler tour và bảng thưa**	O(n log n) chuẩn bị, O(1) mỗi truy vấn, bộ nhớ khoảng 2n log 2n	Họ hàng [HOHANG]
6	**Cấu trúc các tập rời nhau (DSU)**	O(α(n)) trung bình mỗi thao tác	Nhóm học tập [NHOMHOC]
7	**Thuật toán Tarjan ngoại tuyến**	Gần như O(n + q), bộ nhớ O(n + q)	Điểm gặp nhau [GAPNHAU]
8	**Chia nặng nhẹ (Heavy-Light Decomposition)**	O(n)	Tuyến xe buýt [TUYENXE]
9	**LCA bằng chia nặng nhẹ**	O(n) chuẩn bị, O(log n) mỗi truy vấn, bộ nhớ O(n)	Gia phả dòng họ [GIAPHA]
10	**Khoảng cách và đường đi trên cây**	Như phương pháp LCA được dùng	Khoảng cách (CSES Distance Queries) [KHOANGCACH]

Lê Thành Nam

Bài	Kiến thức	Độ phức tạp	Bài vận dụng
11	**Hiệu tiền tố trên cây**	O((n + m) log n) với bảng nhảy	Đếm lượt đi qua (CSES Counting Paths) [DEMLUOT]
12	**Thuật toán Farach-Colton – Bender**	O(n) chuẩn bị, O(1) mỗi truy vấn, O(n) bộ nhớ	Siêu tốc [SIEUTOC]
13	**Tổ tiên thứ k trong O(1): phân rã thang**	O(n log n) chuẩn bị, O(1) mỗi câu hỏi	Cấp trên thứ k, phiên bản siêu tốc [CAPTREN2]

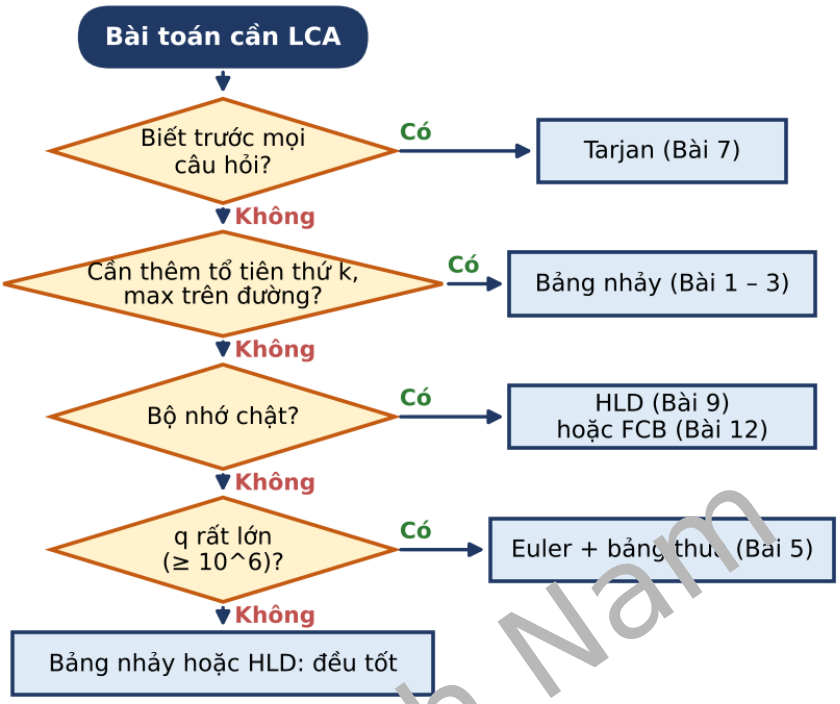
So sánh các phương pháp tìm LCA

Phương pháp	Chuẩn bị	Mỗi truy vấn	Bộ nhớ	Kiểu
Ngây thơ	—	O(n)	O(n)	trực tuyến
Bảng nhảy (Bài 2)	O(n log n)	O(log n)	O(n log n)	trực tuyến
Euler + bảng thưa (Bài 5)	O(n log n)	O(1)	O(n log n)	trực tuyến
Tarjan (Bài 7)	gần O(n + q) cho tất cả		O(n + q)	ngoại tuyến
Chia nặng nhẹ (Bài 9)	O(n)	O(log n)	O(n)	trực tuyến
Farach-Colton – Bender (Bài 12)	O(n)	O(1)	O(n)	trực tuyến

5. Chọn phương pháp nào?

Không có phương pháp "tốt nhất" cho mọi đề. Sơ đồ dưới đây là một gợi ý xuất phát từ giới hạn của đề bài:

Lê Thành Nam



***Hình 0.6** Gợi ý chọn phương pháp*

Trong các kì thi học sinh giỏi, **bảng nhảy** là lựa chọn mặc định: ngắn, dễ nhớ, làm được thêm nhiều việc (tổ tiên thứ k, max/min trên đường). Các phương pháp còn lại là những "vũ khí" khi giới hạn thời gian hoặc bộ nhớ quá chặt, hoặc khi đề cho phép xử lí ngoại tuyến.

6. Phương pháp giải một bài có LCA

Các bài vận dụng trong sách đều giải theo năm bước sau:

NĂM BƯỚC

1. **Đọc dữ liệu, dựng cây:** mảng cha (đề cho "cấp trên trực tiếp") hay danh sách cạnh (cần BFS từ gốc 1 để có par, dep, ord).
2. **Dịch câu hỏi sang ngôn ngữ cây:** tổ tiên thứ k? LCA? khoảng cách? giá trị trên đường đi? cộng lên đường đi?
3. **Chọn phương pháp** theo giới hạn n, q, bộ nhớ, trực tuyến hay ngoại tuyến (Mục 5).
4. **Chạy tay** trên một cây nhỏ (T14 là đủ), so với cách ngây thơ.
5. **Kiểm chứng** bằng chương trình trâu trên hàng trăm cây ngẫu nhiên nhỏ, và chạy thử với cây đường thẳng lớn nhất.

Lê Thanh Nam

Sinh một cây ngẫu nhiên n đỉnh, gốc 1, dạng mảng cha p[2..n]

```
mt19937 rng(seed);
vector<int> perm(n);
iota(perm.begin(), perm.end(), 1);
shuffle(perm.begin() + 1, perm.end(), rng);
vector<int> p(n + 1);
for (int i = 1; i < n; i++)
    p[perm[i]] = perm[rng() % i];
```

Đoạn trên xáo trộn nhãn (giữ đỉnh 1 làm gốc) để cha không nhất thiết có số hiệu nhỏ hơn con: đó là trường hợp dễ làm sai nhất (quên BFS mà duyệt theo số hiệu). Để có cây đường thẳng, thay `rng() % i` bằng `i - 1`.

TÓM TẮT PHẦN MỞ ĐẦU

- LCA(u, v): tổ tiên chung sâu nhất; tồn tại, duy nhất; các tổ tiên chung là LCA và tổ tiên của nó.
- Đường u – v đi qua LCA; d(u, v) = dep[u] + dep[v] − 2·dep[LCA].
- LCA giao hoán, kết hợp; LCA của một tập = LCA của hai đỉnh có tin nhỏ nhất và lớn nhất.
- Ngây thơ O(n) mỗi câu; sáu chương sau đưa về O(log n), O(1), hoặc gần O(1) ngoại tuyến.

Lê Thành Nam

CHƯƠNG 1. BẢNG NHẢY NHỊ PHÂN

Thay vì đi lên từng bậc, ta học cách nhảy 1, 2, 4, 8, ... bậc một lần. Chỉ với một bảng chuẩn bị trước, mọi câu hỏi về tổ tiên, tổ tiên chung và thông tin trên đường đi đều được trả lời trong O(log n).

TRONG CHƯƠNG NÀY

Bài 1. Tổ tiên thứ k và bảng nhảy nhị phân *(vận dụng: cấp trên thứ k (CSES Company Queries I))*

Bài 2. LCA bằng bảng nhảy nhị phân *(vận dụng: sếp chung (CSES Company Queries II))*

Bài 3. Thông tin trên đường đi bằng bảng nhảy *(vận dụng: đoạn đường dốc nhất)*

Lê Thành Nam

BÀI 1 Tổ tiên thứ k và bảng nhảy nhị phân

Bài vận dụng: Cấp trên thứ k (CSES Company Queries I)

TÓM TẮT BÀI 1. TỔ TIÊN THỨ K VÀ BẢNG NHẢY NHỊ PHÂN

Chủ đề: Bảng nhảy nhị phân

Kiến thức chính:

- anc(v, k): đi lên k cạnh; tồn tại khi k ≤ dep[v].
- Viết k ở hệ nhị phân: k bước = các bước nhảy 2^j ứng với bit 1 (thứ tự tùy ý).
- **up[j][v]** = tổ tiên thứ 2^j; up[j][v] = up[j − 1][up[j − 1][v]]; đỉnh 0 = "ngoài cây".

Khuôn cài đặt:

```
for (j = 1; j < LOG; j++)
    for (v = 1; v <= n; v++)
        up[j][v] = up[j - 1][up[j - 1][v]];
for (j = 0; j < LOG; j++)
    if (k >> j & 1) x = up[j][x];
```

Độ phức tạp: Chuẩn bị O(n log n), mỗi truy vấn O(log n), bộ nhớ O(n log n)

Dùng khi: Hỏi tổ tiên thứ k, tổ tiên ở độ sâu d

Bài vận dụng: Cấp trên thứ k (CSES Company Queries I) [CAPTRENK]

Lê Thành Nam

Học xong bài này, em sẽ:

- ✓ Hiểu khái niệm **tổ tiên thứ k** và vì sao đi lên từng bước là quá chậm.
- ✓ Biết cách **viết k ở hệ nhị phân** để thay k bước bằng không quá $\log_2 k$ lần nhảy.
- ✓ Xây **bảng nhảy nhị phân** up[j][v] bằng công thức truy hồi và chứng minh công thức đó.
- ✓ Trả lời câu hỏi "tổ tiên thứ k của v" trong **O(log n)**, kể cả khi tổ tiên đó không tồn tại.

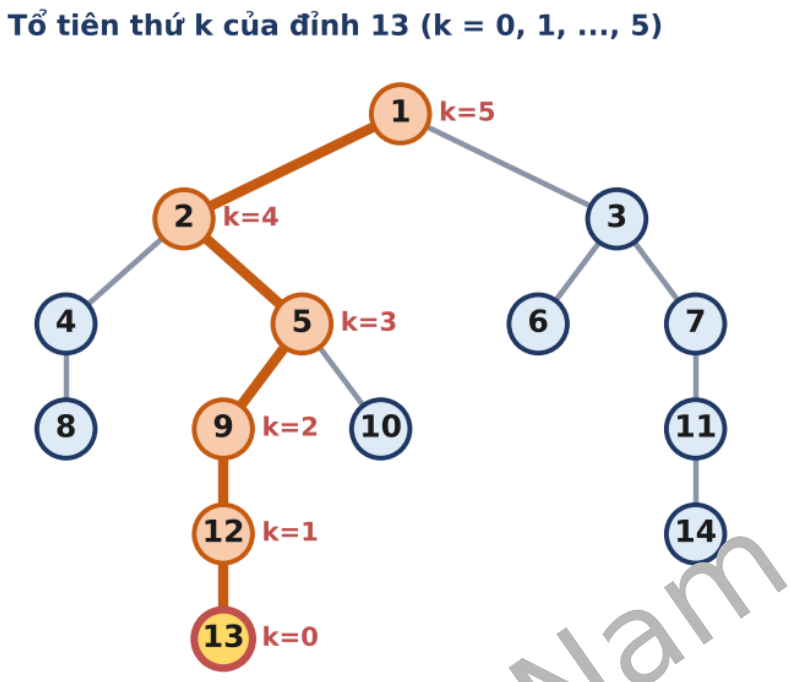
PHẦN A LÍ THUYẾT

A1. Tổ tiên thứ k

Trong cả tập sách, cây luôn được treo ở **gốc 1**; par[v] là cha của v (par[1] = 0), dep[v] là độ sâu của v (dep[1] = 0). Những khái niệm này đã có ở Tập 1, Bài 2.

ĐỊNH NGHĨA

- **Tổ tiên thứ 0** của v là chính v.
- **Tổ tiên thứ k** (k ≥ 1) của v là cha của tổ tiên thứ k − 1 của v. Kí hiệu: **anc(v, k)**.
- Nói cách khác, anc(v, k) là đỉnh ta tới được khi đi từ v lên phía gốc đúng k cạnh.



Hình 1.1. *Các tổ tiên của đỉnh 13*

Trên Hình 1.1: anc(13, 1) = 12, anc(13, 2) = 9, anc(13, 3) = 5, anc(13, 4) = 2, anc(13, 5) = 1. Đi thêm một bước nữa thì ra khỏi cây: anc(13, 6) **không tồn tại**.

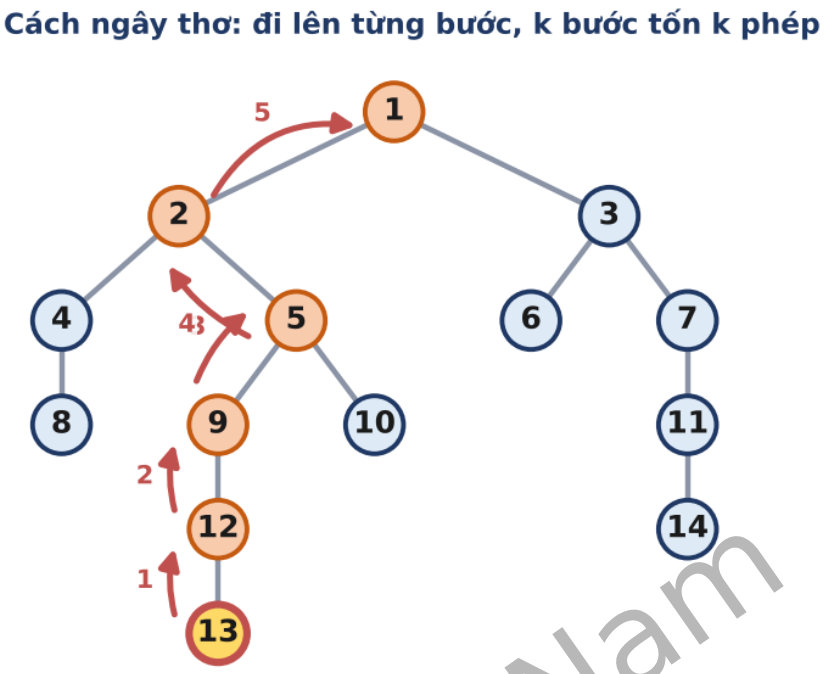
TÍNH CHẤT

- anc(v, k) tồn tại **khi và chỉ khi k ≤ dep[v]**: đường từ v lên gốc chỉ có dep[v] cạnh.
- anc(v, dep[v]) luôn là gốc 1.
- dep[anc(v, k)] = dep[v] − k: mỗi bước đi lên làm độ sâu giảm đúng 1.

Lê Thành Nam

A2. Cách ngây thơ và giới hạn của nó

Cách tự nhiên nhất là làm đúng như định nghĩa: lặp k lần "x = par[x]".



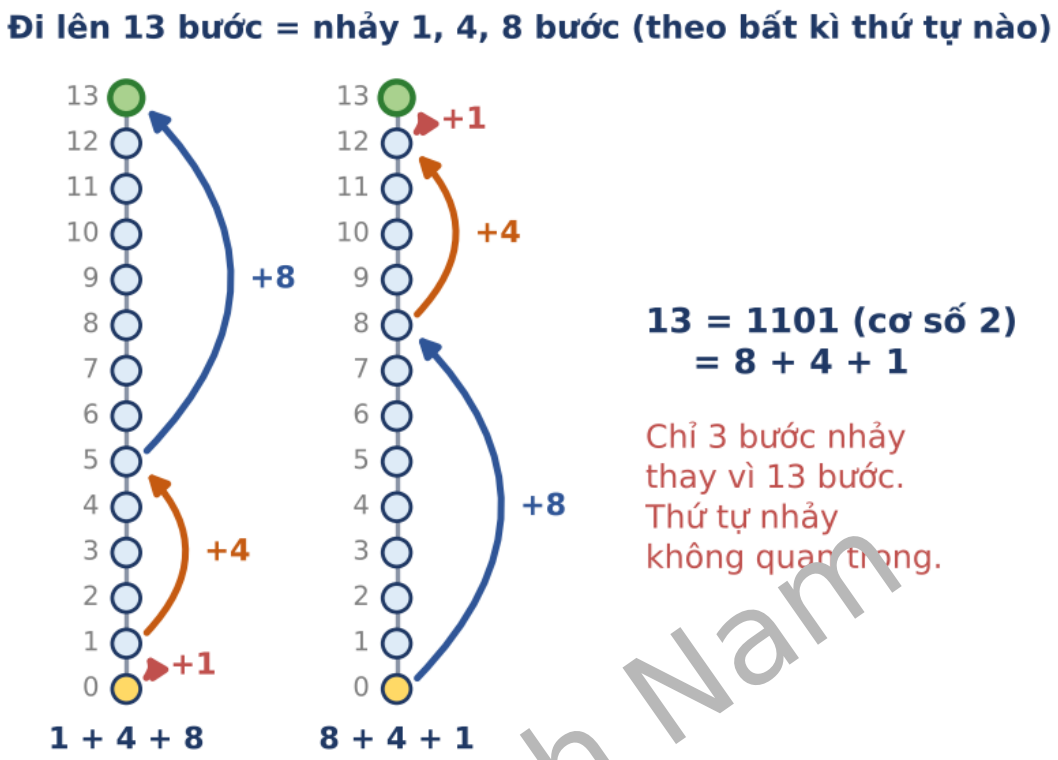
***Hình 1.2.** Đi lên từng bước: anc(13, 5) tốn 5 phép gán*

Mỗi câu hỏi tốn **k** phép gán. Với cây "đường thẳng" n đỉnh, k có thể tới n − 1, nên một câu hỏi tốn O(n). Nếu có q câu hỏi, tổng cộng là O(n·q): với n = q = $2 \cdot 10^5$, đó là $4 \cdot 10^{10}$ phép gán, mất hàng chục giây. Ta cần một cách **đi nhanh hơn từng bước một**.

A3. Ý tưởng: viết k ở hệ nhị phân

Mọi số tự nhiên k đều viết được thành tổng các lũy thừa **khác nhau** của 2, đúng theo các chữ số 1 trong cách viết nhị phân của k. Ví dụ $13 = 1101_2 = 8 + 4 + 1$, nên đi 13 bước cũng chính là "nhảy 8 bước, nhảy 4 bước, nhảy 1 bước".

Lê Thành Nam

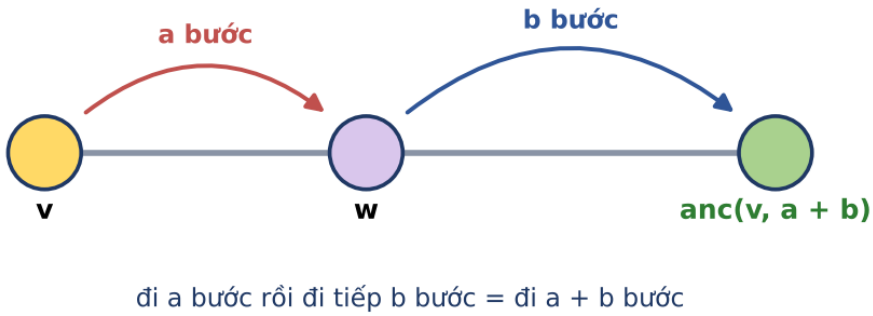


Hình 1.3. *Ba lần nhảy thay cho 13 bước*

BỔ ĐỀ CỘNG BƯỚC

Với mọi a, b ≥ 0: **anc(anc(v, a), b) = anc(v, a + b)** (nếu một trong hai vế không tồn tại thì vế kia cũng không tồn tại).

Ý 1 **Đi a bước rồi đi tiếp b bước**



Hình 1.4. *Đi a bước rồi đi tiếp b bước*

Đi từ v lên a cạnh, tới w = anc(v, a). Từ w đi tiếp lên b cạnh. Trên cây, đường đi lên từ một đỉnh là **duy nhất** (mỗi đỉnh chỉ có một cha), nên hai chặng nối lại chính là đường đi lên a + b

Lê Thành Nam

cạnh từ v. Nếu đường đi lên từ v có ít hơn a + b cạnh thì ở chặng nào đó ta ra khỏi cây, và vế phải cũng không tồn tại.

Ý 2 **Hệ quả: thứ tự nhảy không quan trọng**

Áp dụng bổ đề nhiều lần: nhảy các bước 8, 4, 1 theo thứ tự nào thì tổng số cạnh đã đi vẫn là 13, nên điểm đến như nhau. Ta có thể duyệt các bit của k từ thấp lên cao hoặc từ cao xuống thấp.

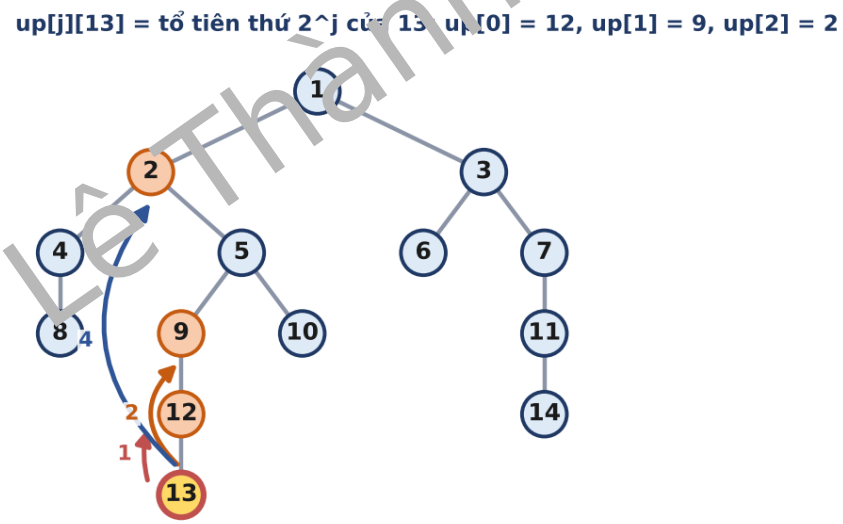
Số chữ số 1 trong k không vượt quá số chữ số nhị phân của k, tức là $\lfloor \log_2 k \rfloor + 1$. Với $k < 2^{18} = 262\,144$, mỗi câu hỏi chỉ cần **tối đa 18 lần nhảy**. Điều kiện là ta phải nhảy được 2^j bước **trong một phép**: cần biết sẵn tổ tiên thứ 1, 2, 4, 8, ... của mọi đỉnh.

A4. Bảng nhảy nhị phân

ĐỊNH NGHĨA

up[j][v] = anc(v, 2^j): tổ tiên thứ 2^j của v. Nếu tổ tiên đó không tồn tại, quy ước up[j][v] = 0.

Bảng có LOG hàng (j = 0, 1, ..., LOG − 1) và n cột (v = 1, ..., n). LOG được chọn sao cho 2^{LOG} lớn hơn mọi k có thể hỏi.



Hình 1.5. *Ba ô của cột 13 trong bảng nhảy*

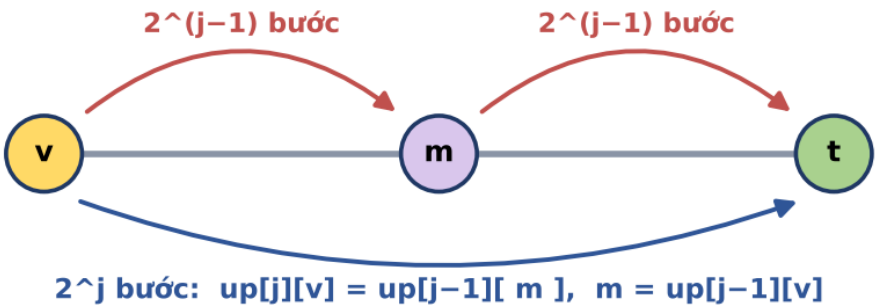
Hàng đầu tiên rất dễ: up[0][v] = anc(v, 1) = **par[v]**. Mấu chốt là tính các hàng sau từ hàng trước.

CÔNG THỨC TRUY HỒI

Với $j \geq 1$: **up[j][v] = up[j − 1][up[j − 1][v]]**.

Lê Thành Nam

Ý 1 Chia 2^j bước thành hai nửa

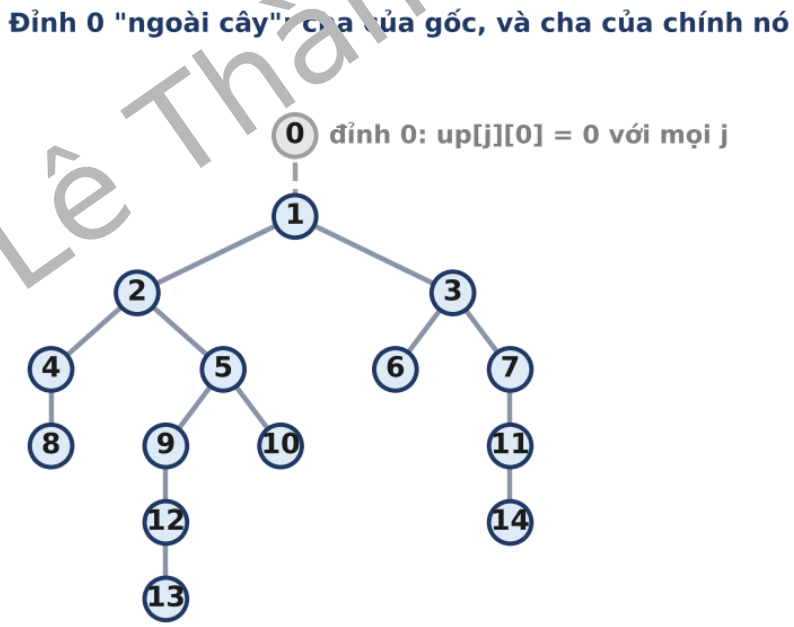


Hình 1.6. *Chia 2^j bước thành hai nửa*

Vì $2^j = 2^{j-1} + 2^{j-1}$, theo bổ đề cộng bước: $anc(v, 2^j) = anc(anc(v, 2^{j-1}), 2^{j-1})$. Đặt m = up[j − 1][v] là điểm giữa; nhảy thêm 2^{j-1} bước từ m chính là up[j − 1][m].

Ý 2 Trường hợp vượt ra ngoài cây

Nếu nửa đầu đã vượt ra ngoài cây thì m = 0. Ta quy ước **up[j][0] = 0 với mọi j** (đỉnh 0 "ngoài cây" nhảy đi đâu cũng vẫn ở ngoài cây), khi đó công thức cho up[j][v] = up[j − 1][0] = 0, đúng như mong muốn. Nếu nửa đầu còn trong cây nhưng nửa sau vượt ra ngoài thì up[j − 1][m] = 0, cũng đúng.



Hình 1.7. *Đỉnh 0 đóng vai trò "ngoài cây"*

Lê Thành Nam

VÌ SAO DÙNG ĐỈNH 0?

Đỉnh 0 giúp code không cần câu lệnh if nào: mảng up có n + 1 cột, cột 0 toàn số 0. Một khi đã rơi ra ngoài cây (giá trị 0), mọi lần nhảy tiếp theo đều giữ nguyên 0. Cuối cùng chỉ cần kiểm tra kết quả có bằng 0 hay không.

A5. Xây bảng, chạy tay từng hàng

Theo công thức, mọi ô của hàng j chỉ cần hàng j − 1. Vì vậy ta tính **lần lượt từng hàng, j tăng dần**; trong một hàng, thứ tự các v không quan trọng (khác với độ sâu, ở đây **không cần** duyệt cha trước con).

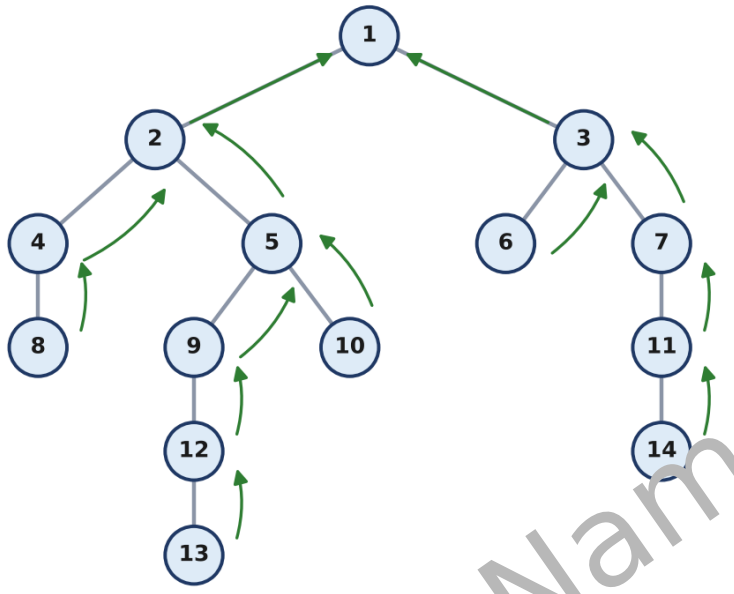
Code mẫu: Xây bảng nhảy

```
for (int i = 2; i <= n; i++) up[0][i] = par[i];
for (int j = 1; j < LOG; j++)
    for (int v = 1; v <= n; v++)
        up[j][v] = up[j - 1][up[j - 1][v]];
```

Lê Thành Nam

Bước nhỏ 1 **Hàng j = 0: chép mảng cha**

Hàng j = 0 chính là mảng cha (gốc 1 có cha là 0)



v	1	2	3	4	5	6	7	8	9	10	11	12	13	14
up[0]	0	1	1	2	2	3	3	4	5	5	7	9	12	11
up[1]														
up[2]														
up[3]														

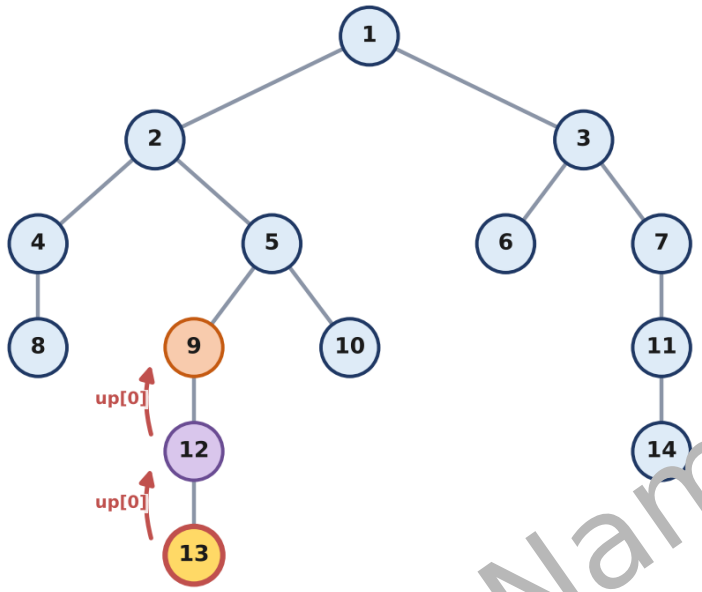
Lê Thành Nam

Hình 1.8. *Hàng j = 0: chép mảng cha*

up[0][v] = par[v]. Gốc 1 có up[0][1] = 0.

Bước nhỏ 2 **Hàng j = 1: nhảy 2 bước**

Hàng j = 1: up[1][13] = up[0][up[0][13]] = up[0][12] = 9



v	1	2	3	4	5	6	7	8	9	10	11	12	13	14
up[0]	0	1	1	2	2	3	3	4	5	5	7	9	12	11
up[1]	0	0	0	1	1	1	1	2	2	2	3	5	9	7
up[2]														
up[3]														

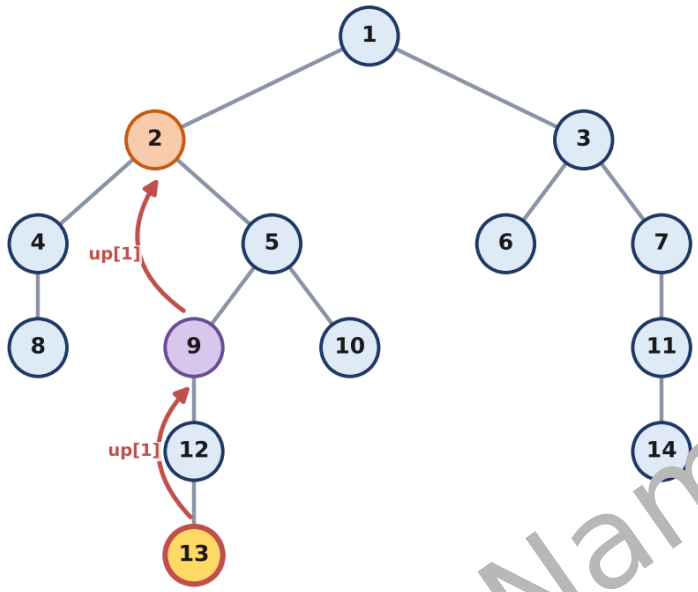
Lê Thành Nam

Hình 1.9. *Hàng j = 1: nhảy 2 bước*

Mỗi ô lấy hai lần hàng 0. Ví dụ ô (1, 13): up[0][13] = 12, rồi up[0][12] = 9. Ô (1, 2): up[0][2] = 1, up[0][1] = 0, nên up[1][2] = 0 (2 chỉ có một tổ tiên thực sự).

Bước nhỏ 3 **Hàng j = 2: nhảy 4 bước**

Hàng j = 2: up[2][13] = up[1][up[1][13]] = up[1][9] = 2



v	1	2	3	4	5	6	7	8	9	10	11	12	13	14
up[0]	0	1	1	2	2	3	3	4	5	5	7	9	12	11
up[1]	0	0	0	1	1	1	1	2	**2**	2	3	5	**9**	7
up[2]	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**1**	**2**	**1**
up[3]														

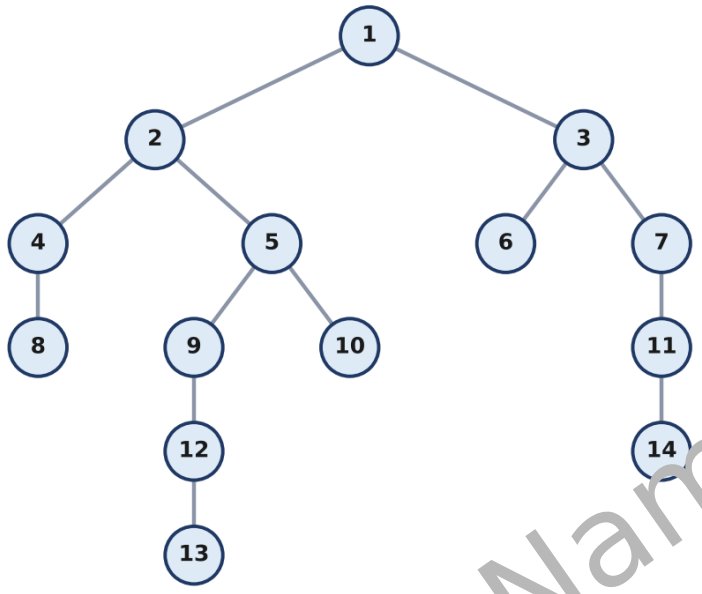
Lê Thành Nam

Hình 1.10. *Hàng j = 2: nhảy 4 bước*

Ô (2, 13): up[1][13] = 9, rồi up[1][9] = 2. Chỉ ba đỉnh 12, 13, 14 có độ sâu ≥ 4 nên chỉ ba ô này khác 0.

Bước nhỏ 4 **Hàng j = 3: nhảy 8 bước**

Hàng j = 3: không đỉnh nào sâu tới 8, cả hàng bằng 0



v	1	2	3	4	5	6	7	8	9	10	11	12	13	14
up[0]	0	1	1	2	2	3	3	4	5	5	7	9	12	11
up[1]	0	0	0	1	1	1	1	2	2	2	3	5	9	7
up[2]	0	0	0	0	0	0	0	0	0	0	0	1	2	1
up[3]	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**

Lê Thành Nam

Hình 1.11. *Hàng j = 3: nhảy 8 bước*

Cây sâu nhất là 5, nên không đỉnh nào có tổ tiên thứ 8: cả hàng bằng 0. Hàng này vẫn cần nếu đề cho phép hỏi k tới 13 (để nhận ra câu trả lời −1).

A6. Trả lời câu hỏi anc(v, k)

NHẢY THEO CÁC BIT CỦA k

1. Đặt x = v.
2. Với j = 0, 1, ..., LOG − 1: nếu bit thứ j của k bằng 1 thì x = up[j][x].
3. Nếu x = 0 thì tổ tiên không tồn tại (in −1); ngược lại x là đáp số.

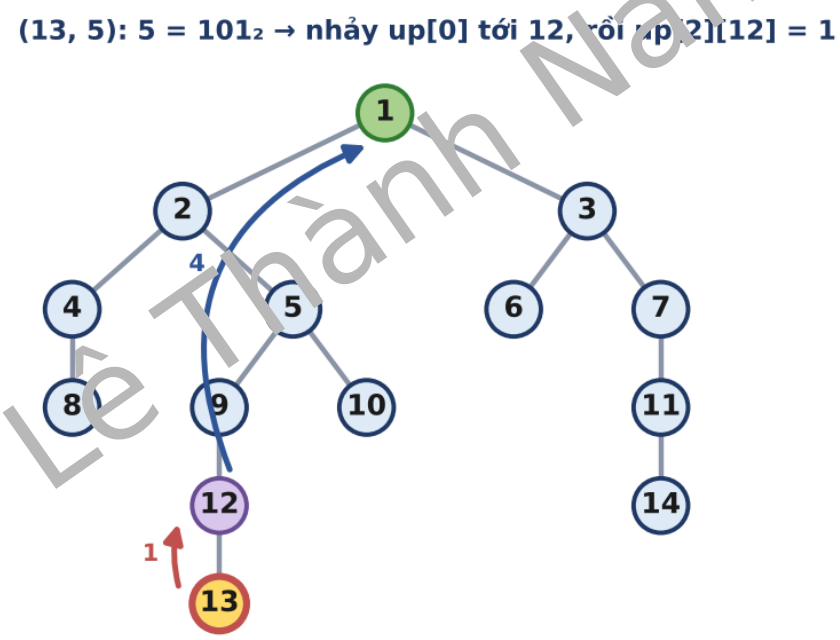
HỘP CÔNG CỤ C++: PHÉP TOÁN BIT

`k >> j` dịch k sang phải j bit (chia nguyên cho 2^j); `(k >> j) & 1` lấy **bit thứ j** của k (bằng 0 hoặc 1). Toán tử >> được thực hiện trước &, nên viết gọn `k >> j & 1`.

`1 << j` bằng 2^j. Ví dụ k = 13 = 1101_2: các bit j = 0, 2, 3 bằng 1; bit j = 1 bằng 0.

k	Nhị phân	Các bước nhảy	Số lần nhảy
5	101	1 + 4	2
6	110	2 + 4	2
13	1101	1 + 4 + 8	3
100 000	11000011010100000	6 lũy thừa của 2	6
262 143	18 chữ số 1	$1 + 2 + ... + 2^{17}$	18

Bước nhỏ 1 **Câu hỏi (13, 5)**



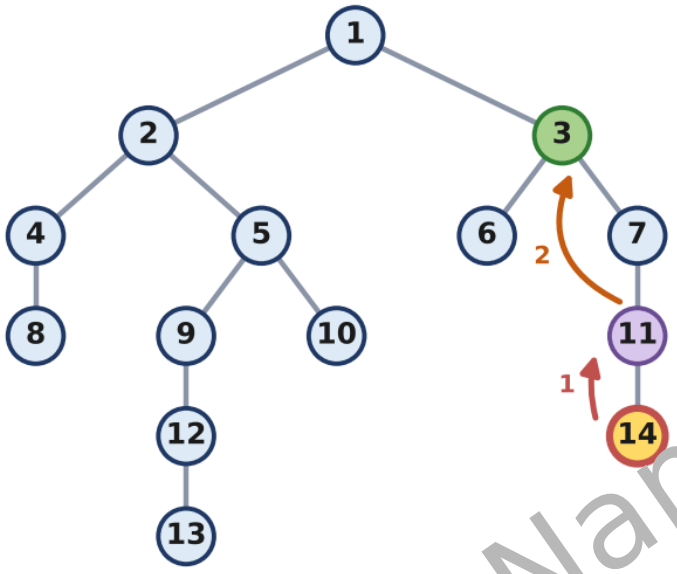
Hình 1.12. *Câu hỏi (13, 5)*

$5 = 101_2$: bit 0 bằng 1, nhảy up[0][13] = 12; bit 1 bằng 0, bỏ qua; bit 2 bằng 1, nhảy up[2][12] = 1. Đáp số **1**, đúng với Hình 1.1.

Lê Thành Nam

Bước nhỏ 2 **Câu hỏi (14, 3)**

(14, 3): 3 = 11_2 → nhảy up[0] tới 11, rồi up[1][11] = 3

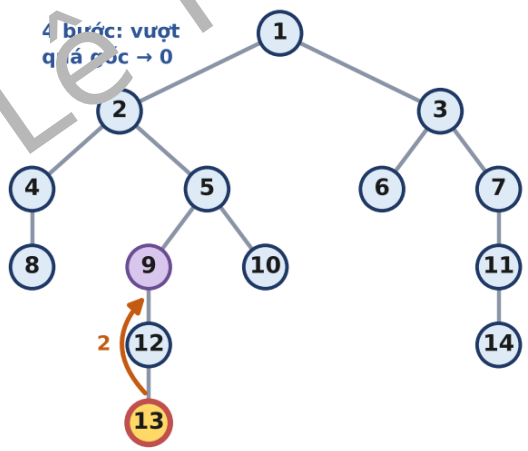


Hình 1.13. *Câu hỏi (14, 3)*

$3 = 11_2$: up[0][14] = 11, rồi up[1][11] = 3. Đáp số 3.

Bước nhỏ 3 **Câu hỏi (13, 6)**

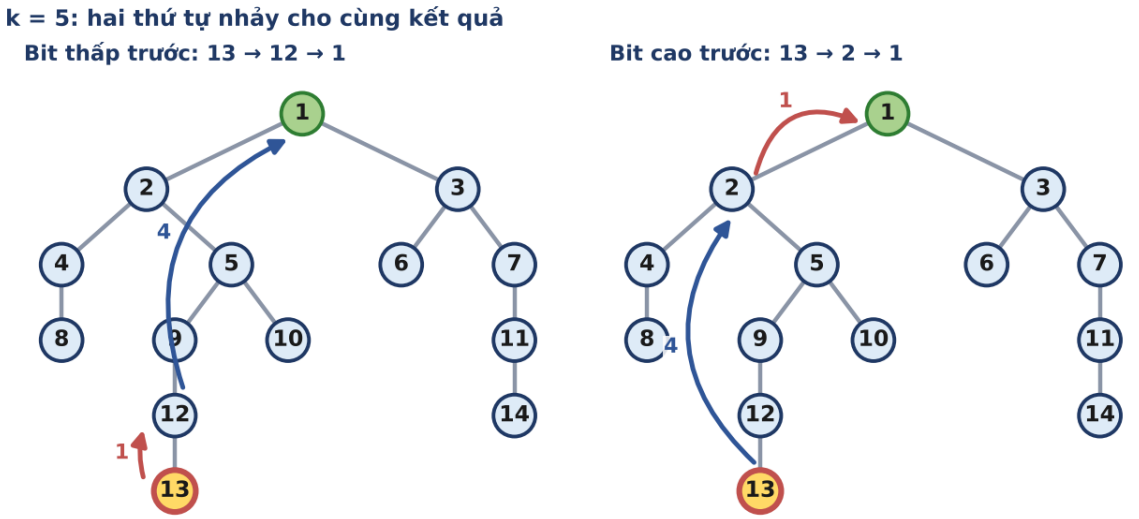
(13, 6): 6 = 11_2 → up[1][13] = 9, rồi up[2][9] = 0: không tồn tại (−1)



Hình 1.14. *Câu hỏi (13, 6)*

$6 = 110_2$: bit 1: up[1][13] = 9; bit 2: up[2][9] = 0 (9 chỉ sâu 3, không có tổ tiên thứ 4). x = 0, in **−1**. Thật vậy 6 > dep[13] = 5.

Lê Thành Nam



Hình 1.15. *Duyệt bit thấp trước hay bit cao trước đều cho cùng đáp số*

A7. Vì sao thuật toán đúng

Ý 1 **Sau khi xét các bit 0..j, x là tổ tiên thứ (k mod 2^(j+1)) của v**

Chứng minh bằng quy nạp theo j. Trước vòng lặp x = v = anc(v, 0). Khi xét bit j: nếu bit bằng 0 thì phần dư k mod 2^{j+1} không đổi và x giữ nguyên; nếu bit bằng 1 thì phần dư tăng thêm 2^j, và x = up[j][x] = anc(x, 2^j), theo bổ đề cộng bước đúng là tổ tiên thứ (phần dư mới) của v.

Ý 2 **Kết thúc vòng lặp**

Khi j chạy hết LOG bit và $2^{LOG} > k$ thì k mod 2^{LOG} = k: x = anc(v, k). Nếu tổ tiên không tồn tại, ở lần nhảy nào đó x rơi về 0 và giữ nguyên 0 tới cuối (up[j][0] = 0).

A8. Chọn LOG và bộ nhớ

LOG là số hàng của bảng, cần **$2^{LOG} > k$ lớn nhất**. Nếu $k \le n - 1$ thì chọn LOG sao cho $2^{LOG} \ge n$ là đủ.

n tối đa	LOG	Số ô LOG × (n + 1)	Bộ nhớ (int, 4 byte)
10^3	10	$\approx 10^4$	40 KB
10^5	17	$1{,}7 \cdot 10^6$	6,8 MB
$2 \cdot 10^5$	18	$3{,}6 \cdot 10^6$	14,4 MB
10^6	20	$2 \cdot 10^7$	80 MB

> **up[j][v] HAY up[v][j]?**
> Hai cách đều đúng. Sách dùng up[j][v] (hàng là j): khi xây bảng, vòng trong chạy theo v nên đọc ghi bộ nhớ liên tục, nhanh hơn; mỗi hàng là một vector riêng nên dễ viết `up[0] = par`.

Lê Thành Nam

ĐẾM SỐ PHÉP TÍNH

- Xây bảng: LOG·n phép gán = **O(n log n)**.
- Mỗi câu hỏi: LOG lần kiểm tra bit = **O(log n)**.
- Tổng: O((n + q) log n). Với n = q = $2 \cdot 10^5$: khoảng $7 \cdot 10^6$ phép, rất nhanh.

BẪY THƯỜNG GẶP

- LOG quá nhỏ: với $k \geq 2^{LOG}$, các bit cao của k bị bỏ qua, đáp số sai mà không báo lỗi.
- Đặt up[0][1] = 1 (gốc là cha của chính nó) thay vì 0: khi đó không phân biệt được "dừng ở gốc" với "vượt quá gốc". (Ở Bài 2 ta lại dùng quy ước này một cách có chủ ý.)
- Tính hàng j trước khi hàng j − 1 xong: phải để vòng j ở ngoài, vòng v ở trong.
- Dùng `1 << j` với j ≥ 31: tràn số. Với LOG ≤ 20 thì an toàn.

GHI NHỚ

- up[j][v] = tổ tiên thứ 2^j của v; up[0] = par; up[j][v] = up[j − 1][up[j − 1][v]].
- anc(v, k): nhảy up[j] với mỗi bit j bằng 1 của k; rơi về 0 nghĩa là không tồn tại.
- Xây O(n log n), hỏi O(log n), bộ nhớ LOG·n.

PHẦN B BÀI VẬN DỤNG: CẤP TRÊN THỨ K

Áp dụng kiến thức Phần A để giải một bài toán theo khuôn đề thi [CAPTRENK].

Lê Thành Nam